

Python Strings Summary

1. `str()` Function and Its Use Cases

The `str()` function converts an object to its string representation.

Use Cases:

1. Comparing if 2 numbers are palindrome:

```
def palindrome(number):  
    return str(number) == str(number)[::-1]  
print(palindrome(121)) # True
```

2. String Indexing, Slicing, and Stride

Indexing:

Python strings are indexed from `0` (positive indexing) and `-1` (negative indexing).

Example:

```
s = "Python"  
print(s[0]) # The first character 'P'  
print(s[-1]) # The last character 'n'
```

Slicing:

Extract parts of a string using slicing: `string[start:end]`. Note: The end is **exclusive**.

Example:

```
print(s[1:4]) # 'yth'  
print(s[:3]) # 'Pyt'  
print(s[3:]) # 'hon'
```

Stride:

Use a stride to skip characters: `string[start:end:step]`.

Example:

```
print(s[::2]) # 'Pto' (Selecting even indices only)
print(s[::-1]) # 'nohtyP' (reversing a string)
```

3. String Methods and Practical Use Cases

Common Methods:

1. `lower()` and `upper()`:

```
text = "Hello World"
print(text.lower()) # 'hello world'
print(text.upper()) # 'HELLO WORLD'
```

Use Case: Normalize user input for case-insensitive comparison.

2. `isupper()` and `islower()`:

```
text = "HELLO"
print(text.isupper()) # True

text = "hello"
print(text.islower()) # True
```

Use Case: Validate if user input is in the expected case.

3. `replace()`:

```
text = "I like Python."
print(text.replace("Python", "coding")) # 'I like coding.'
```

Use Case: Replace sensitive data or update content.

4. `split()` and `join()`:

```
text = "apple,banana,cherry"
fruits = text.split(",")
print(fruits) # ['apple', 'banana', 'cherry']

sentence = " ".join(fruits)
print(sentence) # 'apple banana cherry'
```

Use Case: Parsing CSV data or constructing sentences.

5. `startswith()` and `endswith()`:

```
text = "example.txt"
print(text.endswith(".txt")) # True
print(text.startswith("ex")) # True
```

Use Case: Validate file types or URLs.

6. `find()` and `index()`:

```
text = "Python programming"
print(text.find("prog")) # 7
print(text.index("prog")) # 7
```

Use Case: Locate substrings in larger texts.

7. `count()`:

```
text = "banana"
print(text.count("a")) # 3
```

Use Case: Count character frequency.

8. `isnumeric()`, `isalpha()`, `isalnum()`:

```
print("123".isnumeric()) # True
print("abc".isalpha()) # True
print("abc123".isalnum()) # True
```

Use Case: Validate user input.

9. `swapcase()`:

```
print('Hello World'.swapcase()) # hELLO wORLD
```

Use Case: Swap cases of the string.

4. Counting Character Frequencies Using **Counter**

Example:

```
from collections import Counter

text = "hello world"
char_count = Counter(text)
print(char_count) # Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, ' ': 1, 'w': 1, 'r': 1, 'd': 1})
```

Use Cases:

1. Finding the Most Frequent Character:

```
most_common = char_count.most_common(1)
print(most_common) # [('l', 3)]
```

2. Detecting Anagrams: (If 2 strings contains the same letters but with different order)

```
def are_anagrams(str1, str2):
    return Counter(str1) == Counter(str2)

print(are_anagrams("listen", "silent")) # True
```

3. Detecting if this string can form a palindrome: (Read from left the same as right)

```
# Count how many characters have odd frequencies
odds_count = 0
for i in char_count.values():
    if i % 2 == 1:
```

```

        odds_count += 1

# We can form a palindrome if at most one character has
an odd frequency
if odds_count <= 1:
    print('Yes')
else:
    print('No')

```

5. `string` Module Variables and Importance

The `string` module contains predefined constants useful for string manipulation.

Constants:

- `string.ascii_letters`: All ASCII letters (lowercase + uppercase).
- `string.ascii_lowercase`: Lowercase letters.
- `string.ascii_uppercase`: Uppercase letters.
- `string.digits`: Digits `0-9`.
- `string.punctuation`: Common punctuation marks (!, @, #, \$, %, ^, &, *, etc.).

Use Cases:

1. Suggesting Secure Passwords:

```

import random
import string

all_characters = string.ascii_letters + string.digits +
string.punctuation

password = "".join(random.choices(all_characters, k=16))
print(password) # &aCRg9B@51F+_~/X

```

2. Detecting pangrams: (If the word contains all Alphabetic letters)

```
from string import ascii_lowercase

t = int(input())
word = set(input().lower())

if len(ascii_lowercase) == len(word):
    print('YES')
else:
    print('NO')
```

6. Top String Problem-Solving Use Cases

1. Reversing a String:

```
text = "hello"
reversed_text = text[::-1]
print(reversed_text)  # 'olleh'
```

2. Checking for Palindromes:

```
def is_palindrome(s):
    return s == s[::-1]

print(is_palindrome("racecar"))  # True
```

3. Finding Duplicates in a String:

```
from collections import Counter

text = "programming"
duplicates = []
for char, count in Counter(text).items():
    if count > 1:
        duplicates.append(char)
print(duplicates)  # ['r', 'g', 'm']
```

4. Removing Duplicates from a String:

```

text = 'programming'
clear_of_duplicates = ''

for i in text:
    if i not in clear_of_duplicates:
        clear_of_duplicates += i

print(clear_of_duplicates)

```

5. Finding the First Non-Repeating Character:

```

def first_unique(s):
    counts = Counter(s)
    for char in s:
        if counts[char] == 1:
            return char
    return None

print(first_unique("swiss")) # 'w'

```

6. Counting Vowels and Consonants:

```

def count_vowels_and_consonants(word: str):
    vowels = "aeiou"
    v_count = 0; c_count = 0
    word = word.replace(' ', '')

    for i in word.lower():
        if i in vowels:
            v_count += 1
        elif i not in vowels:
            c_count += 1

    return v_count, c_count

print(count_vowels_and_consonants("hello world")) # (3,
7)

```

7. Checking Substring Presence:

```
text = "hello world"
print("world" in text) # True
```

8. Sorting Characters in a String:

```
text = "dcba"
sorted_text = "".join(sorted(text))
print(sorted_text) # 'abcd'
```

9. Finding Longest Word in a Sentence:

```
sentence = "The quick brown fox"
longest = max(sentence.split(), key=len)
print(longest) # 'quick'
```

10. Replacing Multiple Substrings:

```
text = "I love Java and JavaScript"
replacements = {"Java": "Python", "JavaScript": "TypeScript"}

for old, new in replacements.items():
    text = text.replace(old, new)

print(text) # 'I love Python and TypeScript'
```

11. Balancing Parentheses:

```
def is_balanced(s):
    stack = []
    for char in s:
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack:
                return False
```



```
        stack.pop()
    return not stack

print(is_balanced("(()))")) # True
print(is_balanced("(()))") # False
```

12. Binary Number to Decimal Conversion:

```
binary = "1010"
decimal = int(binary, 2)
print(decimal) # 10
```

13. Compressing Strings (Run-Length Encoding):

```
def compress_string(s):
    compressed = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            compressed.append(s[i-1] + str(count))
            count = 1
    compressed.append(s[-1] + str(count))
    return ''.join(compressed)

print(compress_string("aaabbc")) # 'a3b2c1'
```